

RL Fine-Tuning of Vision–Language Models for Visual MCQ Reasoning

Course Project Report

Anish A. Kale
Bourns College of Engineering
University of California, Riverside
Riverside, USA
akale014@ucr.edu

Anant Patil
Bourns College of Engineering
University of California, Riverside
Riverside, USA
apati087@ucr.edu

Kedar Shiralkar
Bourns College of Engineering
University of California, Riverside
Riverside, USA
kshir016@ucr.edu

Abstract—Reinforcement learning has recently emerged as an effective mechanism for aligning large multimodal models with task-specific objectives, yet its application to image captioning remains limited by high compute cost and a lack of proper reward and objective design. This project investigates whether reinforcement learning can improve image captions by using a sparse LLM-generated Question-answer accuracy-based reward to overcome the traditional human-biased objectives, as targeted by the recent paper ‘CapRL’ presented at CVPR 2025. Since CapRL’s dataset and code have not yet been made public, we have curated our own. ShareGPT4V captions are used as rich semantic anchors, from which a synthetic multiple-choice question (MCQ) dataset is constructed to provide a downstream reward signal for evaluating caption usefulness. Three policy-gradient algorithms—PPO, GRPO, and REINFORCE—are applied to fine-tune Qwen2-VL-2B-Instruct under a Supervised Learning followed by Reinforcement Learning with Verifiable rewards paradigm.

GRPO achieves the strongest and most stable gains under restricted compute, whereas PPO displays a notable misalignment between its surrogate optimization objective and the true task reward: despite steadily decreasing negative log-likelihood, PPO gradually collapses toward overly brief, under-informative captions that degrade MCQ accuracy. REINFORCE suffers from high-variance gradients and a well-documented form of reward hacking, leading to textual degeneration and unstable policies. These findings demonstrate that caption-focused RL alignment is feasible for small open VLMs, while highlighting fundamental methodological challenges involving sparse rewards, model overconfidence, unintended objective optimization, and the fragility of naïve policy-gradient methods.

Index Terms—Reinforcement learning, vision–language models, image captioning, PPO, GRPO, REINFORCE, multiple-choice question answering.

I. INTRODUCTION

A. Motivation and Background

Vision–language models (VLMs) increasingly support detailed image captioning and multimodal reasoning, yet many models still exhibit shallow grounding: responses may rely more on language priors and dataset biases than on the actual visual evidence in the image. For safety-critical or evaluation-style tasks, such as visual question answering (VQA) or

multiple-choice exams, this lack of grounding can produce confident but incorrect predictions.

In purely textual domains, reinforcement learning (RL)—especially Reinforcement Learning from Human Feedback (RLHF) and its variants—has emerged as a powerful tool for steering large language models toward behavior preferred by humans or external reward functions. By repeatedly sampling outputs and updating the policy based on reward, these methods can bias models towards helpfulness, harmlessness, and honesty.

Current state-of-the-art captioning models are typically trained with Supervised Fine-Tuning (SFT), a paradigm that relies on expensive, non-scalable data annotated by humans or proprietary models. This approach often leads to models that memorize specific ground-truth answers, limiting their generality and ability to generate diverse, creative descriptions. There are methods in the RL domain like RL with imitation terms, LLM-as-a-judge reward-based RL, but all these paradigms come with the risk of reward hacking, a paradigm where the VLM exploits the LLM’s weaknesses to maximize rewards without generating meaningful outputs.

CapRL introduces a robust objective reward term for this subjective task: instead of rewarding captions directly via human preference scores, the model is rewarded for generating captions that enable downstream multiple-choice question answering (MCQ). A caption is considered “good” if it allows a separate evaluator model to answer MCQs correctly. This formulation implicitly favors captions that preserve fine-grained visual details and relationships.

However, the original CapRL system is difficult to reproduce. It requires (a) a proprietary two-million–image dataset, and (b) several 72B-parameter models for captioning, MCQ generation, filtering, and evaluation. These factors make it unsuitable as-is for a graduate classroom project or for research groups constrained to commodity GPUs.

B. Problem Overview

The central question in this project is which RL models (GRPO, PPO, REINFORCE) can contribute in improving the

caption generation quality of smaller VLMs to the state-of-the-art level. Specifically, the aim is to determine whether:

- fully open datasets (ShareGPT4V captions),
- lightweight MCQ generation using smaller instruction-tuned models,
- a compact 2B-parameter VLM (Qwen2.5-VL-3B), and
- accessible compute resources (L4, A100 GPUs)

are sufficient to obtain measurable gains in visual MCQ answering through reinforcement learning.

In the adapted formulation, the objective is not to improve caption generation directly. Instead, the model is trained to select the correct answer for a four-option MCQ given an image and the question. The reward structure is naturally that of a single-step contextual bandit: the model receives a reward of 1 for a correct answer and 0 for an incorrect one. Despite the lack of temporal structure, policy-gradient algorithms such as PPO, GRPO, and REINFORCE can still be applied, and their stability properties in this sparse-reward setting become an object of study in their own right.

C. Contributions

The work makes the following concrete contributions:

- **Open CapRL-inspired pipeline.** A complete, fully open-source pipeline is designed that replaces all proprietary CapRL components with publicly available models and datasets. The pipeline is constrained to be executable within Colab Pro limits.
- **Synthetic MCQ dataset construction.** A dataset is built by pairing ShareGPT4V images and captions with automatically generated MCQs. Each sample contains an image path, a caption, a question, four answer options, and a ground-truth label.
- **RL fine-tuning of Qwen2.5-VL-3B.** The compact Qwen2.5-VL-3B vision-language model is fine-tuned using three policy-gradient algorithms: REINFORCE, PPO, and GRPO. All algorithms operate on the same MCQ dataset and reward definition.
- **Systematic analysis of sparse-reward training.** The project offers an empirical comparison of the three algorithms, including reward curves, NLL trends, and qualitative outputs, and discusses the observed instability of REINFORCE, the overfitting behavior of PPO, and the relative robustness of GRPO.

II. METHODOLOGY

This section reproduces the long-form methodology and adds clarifying details for the RL formulation and implementation.

A. 2.1 Original Plan: CapRL Reproduction Goal

The initial objective was to reproduce, as faithfully as possible, the core behavior of CapRL: improve visual grounding by rewarding downstream MCQ reasoning performance. Since direct reproduction of the official CapRL pipeline was impossible due to model and data constraints, the main design challenge was to extract the essential mechanism—“reward

captions (or policies) that support correct visual reasoning”—and instantiate it using only open resources and small models.

The project therefore treats CapRL as a conceptual blueprint rather than a strict implementation target. The evaluation focuses on whether the adapted pipeline exhibits the same qualitative behavior: does reinforcement learning move the model toward better visual reasoning, or does it merely overfit to artifacts of the reward signal?

B. 2.2 CapRL Methodology (Original Framework)

CapRL’s complete workflow consists of four interacting components:

- 1) **Captioning model.** A large vision-language model generates a rich caption describing the input image. The caption is expected to contain sufficient detail for downstream reasoning.
- 2) **MCQ generator.** A 72B-parameter language model takes the caption as input and generates multiple-choice questions and answers that are entailed by or logically related to the caption.
- 3) **Evaluator model.** A separate high-capacity model reads the original image and the generated MCQ and attempts to answer the question.
- 4) **RL optimizer.** A policy-gradient algorithm modifies the captioning model based on whether the evaluator answers correctly. High evaluator accuracy implies better captions and yields a positive reward.

The key idea is that the caption is rewarded indirectly through its ability to support correct MCQ answering. This formulation encourages captions that preserve visual details and disfavor hallucinated or underspecified descriptions, because such captions would make the evaluator’s job harder.

C. 2.3 Adapted CapRL-Inspired Pipeline (Our Version)

To remain faithful to CapRL while respecting academic compute limits, the pipeline was redesigned as follows:

- 1) **ShareGPT4V Captions.** Instead of generating captions using a massive model online, detailed multi-sentence captions are taken directly from the ShareGPT4V dataset. This provides high-quality semantic descriptions for a large number of images at essentially zero additional compute cost.
- 2) **Lightweight MCQ Generation.** A 7B instruction-tuned language model is used offline to generate four-option MCQs from each caption. For each caption, the model is prompted to produce a natural-language question, four answer options labeled A–D, and a single correct answer among them. The prompts restrict the question to information that can be inferred from the caption.
- 3) **Synthetic MCQ Dataset.** All generated instances are stored as tuples:

(image_path, caption, question, options A–D, correct label).

Minimal heuristic filtering is applied to drop malformed or ambiguous items (e.g., missing options or duplicated correct labels).

- 4) **Policy Model (Qwen2.5-VL-3B).** The 3B-parameter Qwen2.5-VL model serves as the policy network. It receives an image and the question text as input and must select one of the four answer options.
- 5) **Reward Definition.** The reward is defined purely in terms of answer correctness:

$$r = \begin{cases} 1, & \text{if predicted option equals the ground-truth label,} \\ 0, & \text{otherwise.} \end{cases}$$

This yields a sparse, non-differentiable signal dependent on discrete choices.

- 6) **RL Algorithms: PPO, GRPO, and REINFORCE.** Three policy-gradient algorithms are used—PPO, GRPO, and REINFORCE—to explore their relative behavior under identical reward conditions.

This adapted pipeline maintains conceptual fidelity to CapRL—rewarding visual reasoning rather than raw text probability—while being fully open-source and computationally manageable.

D. 2.4 Methodology Diagram (Textual Representation)

The overall flow can be summarized as:

Image dataset (ShareGPT4V) → Rich captions → Lightweight MCQ generation (Question + Options A–D + Correct Label) → Synthetic visual MCQ dataset → Qwen2.5-VL-3B policy model → RL training (PPO / GRPO / REINFORCE) with correctness rewards → Updated policy optimized for visual MCQ accuracy.

This high-level diagram corresponds directly to the scripts and data folders in the project repository.

E. 2.5 Training Policy: Exact Algorithmic and Mathematical Formulation

1) *Environment Definition:* Each MCQ forms a one-step environment:

- **State s .** The state consists of the image features encoded by Qwen2.5-VL and the tokenized MCQ prompt. In practice, the raw image and text are passed into the model, which constructs an internal multimodal representation.
- **Action a .** The action is the selection of one of four discrete choices $a \in \{A, B, C, D\}$.
- **Reward r .** As defined above, $r = 1$ if the selected option matches the ground-truth correct answer, and $r = 0$ otherwise.
- **Episode.** Episodes terminate immediately after one action, so there is no temporal credit assignment problem. The task is therefore a contextual bandit problem rather than a full Markov decision process.

2) *Policy Parameterization:* The policy $\pi_\theta(a | s)$ is represented by the Qwen2.5-VL network with an additional linear head projecting the final hidden state into four logits (z_A, z_B, z_C, z_D) . A softmax layer converts these logits into probabilities over actions.

3) *REINFORCE:* The simplest algorithm considered is REINFORCE. For a single-step episode, the gradient estimator reduces to

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [r \nabla_\theta \log \pi_\theta(a | s)]. \quad (1)$$

In practice, this gradient is approximated using mini-batches of MCQs. Because the reward is either 0 or 1 and there is no baseline, the estimator has high variance, which can lead to unstable and noisy updates.

4) *PPO:* To improve stability, Proximal Policy Optimization (PPO) constrains the size of each update by clipping the probability ratio between the new and old policies. Let $\pi_{\theta_{\text{old}}}$ denote the policy before the current update. For each sampled state–action pair (s_t, a_t) , define the ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (2)$$

Let A_t be an advantage estimate, which, in this single-step setting, is often taken as the reward minus an exponential-moving-average baseline. The PPO objective is

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E} [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]. \quad (3)$$

The clipping prevents r_t from straying too far from 1.0, thereby keeping updates conservative and reducing the risk of catastrophic policy collapse.

5) *Group Relative Policy Optimization (GRPO):* In our setting, GRPO is applied not by sampling multiple captions per image, but by treating each minibatch as a *group*. For each training step, the VLM generates one caption per image in the batch, and a reward is obtained by querying the reward LLM on the corresponding MCQ. GRPO then proceeds as follows:

- 1) For each image in the batch, generate a caption using the current policy.
- 2) Compute a scalar reward r_i for each caption using the reward LLM.
- 3) Normalize rewards within the batch:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon},$$

where μ_r and σ_r are the batch mean and standard deviation.

- 4) Compute the per-sample negative log-likelihood (NLL) of the generated captions.
- 5) Optimize the GRPO objective:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \hat{A}_i \text{NLL}_i,$$

where higher-than-average reward captions receive negative loss (reinforcement) and lower-than-average reward captions receive positive loss (suppression).

Unlike PPO, GRPO does *not* require sampling multiple candidates per prompt or maintaining a KL penalty. The algorithm relies solely on relative performance within the group, making it well-suited for sparse, noisy, or binary reward settings such as MCQ-based caption evaluation.

F. 2.6 Our Implementation

Each RL algorithm is implemented in a dedicated training script:

- `train_reinforce.py`: classical REINFORCE loop without baselines.
- `train_ppo2.py`: PPO variant with moderate clipping (ϵ) and entropy regularization.
- `train_ppo3.py`: PPO variant emphasizing more aggressive entropy regularization and gradient accumulation across multiple mini-batches.
- `train_grpo.py` and `train_grpo_kedar.py`: GRPO implementations differing in how candidate answers are sampled and how group sizes K are chosen.

Each script follows a similar structure: load an MCQ batch, perform a forward pass through Qwen2.5-VL, sample or greedily select an action, compute the reward, compute the loss specific to the algorithm, backpropagate, update the model parameters using AdamW, and log metrics to JSON Lines files.

G. 2.7 Changes to MCQ Generation Pipeline and Reasons

CapRL’s MCQ generation pipeline uses a 72B-parameter QnA generator followed by a second 72B-parameter filter model to ensure question quality and diversity. This was infeasible in our setting for several reasons:

- **Hardware limitations.** Colab Pro cannot reliably host 72B-parameter models with sufficient context length for caption-based question generation.
- **Time constraints.** Training and inference with such large models would greatly exceed the time budget of a course project.
- **Model capacity mismatch.** The target policy model is only 3B parameters; extremely complex questions may not be appropriate for its capacity.

Therefore, the pipeline was simplified to a single 7B instruction-tuned model, i.e. mistralai/Mistral-7B-Instruct-v0.1, prompted carefully to produce straightforward but informative MCQs. Light heuristic filtering (e.g., discarding malformed outputs) replaces the heavy filtering used in CapRL.

H. 2.8 Architecture Diagrams of RL Scripts (Textual)

1) `train_ppo2.py`:

Load MCQ batch $\rightarrow$ Run Qwen2-VL forward pass (image + text) $\rightarrow$ Obtain logits and probabilities over options $\rightarrow$ Sample actions $\rightarrow$ Compute rewards and moving-average baseline $\rightarrow$ Compute advantages A_t $\rightarrow$ Evaluate PPO clipped loss $\rightarrow$ Backpropagate and update parameters $\rightarrow$ Log reward and loss statistics.

2) `train_ppo3.py`: The structure mirrors `train_ppo2.py` but:

- uses larger mini-batch sizes per update,
- accumulates gradients over multiple steps before applying an optimizer step, and
- increases entropy regularization to encourage exploration for longer.

TABLE I
MCQ ACCURACY AND CAPTION LENGTH ACROSS RL ALGORITHMS.

Model	Accuracy	Correct / Total	Avg. Len.
PPO	59.0%	59 / 100	16.2 words
GRPO	63.0%	63 / 100	60.0 words
REINFORCE	30.0%	30 / 100	1.0 word

3) `train_grpo.py`:

Load one MCQ batch $\rightarrow$ Generate K sampled captions using Qwen2-VL $\rightarrow$ Evaluate each caption using a reward LLM to obtain K scalar rewards $\rightarrow$ Normalize rewards within the group to compute advantages via group-wise standardization $\rightarrow$ Apply a REINFORCE-style objective where each caption’s token-level NLL is weighted by its group-normalized advantage $\rightarrow$ Update the policy and log per-step statistics.

4) `train_grpo_kedar.py`: This variant modifies the sampling strategy to use deterministic top- K candidate logits and slightly different reward normalization, but follows the same relative-advantage update rule.

5) `train_reinforce.py`:

Load MCQ batch $\rightarrow$ Forward pass $\rightarrow$ Sample action for each item $\rightarrow$ Compute binary rewards $\rightarrow$ Compute REINFORCE loss $-r \log \pi(a|s)$ $\rightarrow$ Update parameters $\rightarrow$ Log stepwise mean reward and NLL loss.

I. 2.9 Output Directory Explanation

Each run writes outputs in its respective directory under an `output/` hierarchy. Common files include:

- `training_log.jsonl`: one JSON object per step containing step index, reward, mean reward, baseline estimate, NLL loss, and other diagnostics.
- `best/trainer_state.json`: a small JSON file storing the best observed mean reward and the corresponding training step.
- Model checkpoints in HuggingFace format.
- PNG plots (reward curves and loss curves) generated by post-processing notebooks.

III. RESULTS

A. Quantitative Accuracy

Table I reports the final MCQ accuracy for each RL algorithm measured on a held-out test set of 100 MCQs, along with the average length of the generated answer text.

GRPO achieves the highest accuracy, suggesting that its group-relative objective helps the policy find a more robust decision boundary in this sparse-reward setting. PPO performs reasonably well but underperforms GRPO by four percentage points and shows signs of overfitting in its training curves. REINFORCE, exposed directly to the high-variance policy-gradient estimator, fails to achieve competitive performance due to reward hacking. The base model, meanwhile gives an

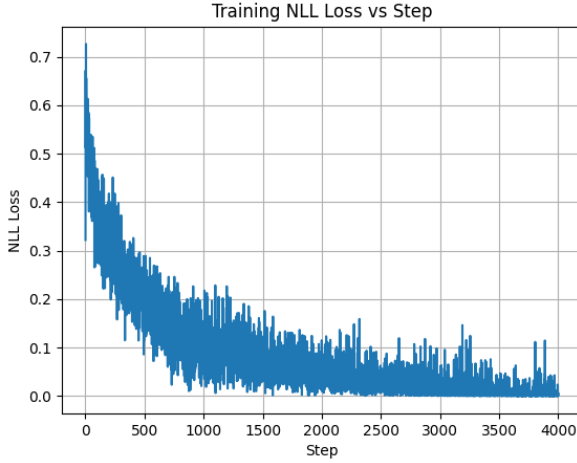


Fig. 1. PPO training NLL loss vs. step.

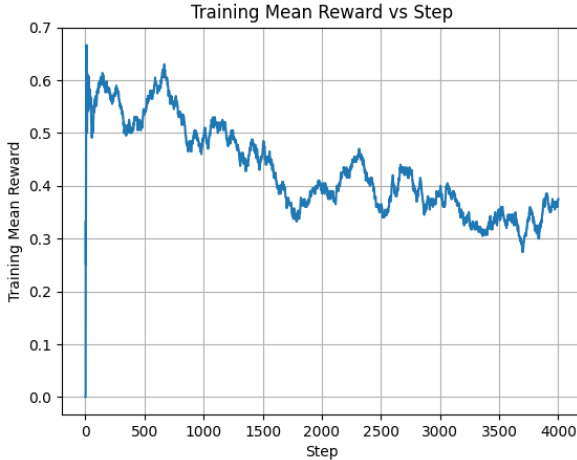


Fig. 2. PPO training mean reward vs. step.

accuracy of around 50%-60% in comparison when tested on the same sample set.

B. PPO Training Behavior and Interpretation

To better understand PPO’s behavior, three plots were generated from the PPO training logs.

Figure 1 shows that the negative log-likelihood (NLL) of the chosen actions decreases smoothly over training. The curve exhibits a rapid initial drop followed by a slower, monotonic decline toward very small values. This indicates that PPO successfully makes the policy more confident and consistent in its chosen answers. From a pure maximum-likelihood perspective, this would typically be interpreted as a sign that the model is fitting the training data well.

However, Figure 2 presents a more nuanced picture: the training mean reward, averaged over sliding windows of steps, initially rises into the 0.55–0.60 range but then slowly drifts downward toward 0.35–0.40. This indicates that although the



Fig. 3. PPO evaluation mean reward vs. step.

model grows more confident (as shown by the decreasing NLL), the actions it becomes confident about are not necessarily the actions that yield high reward. Over time, PPO seems to push the policy into a regime where it makes more deterministic but systematically biased choices.

Figure 3, which plots evaluation mean reward as a function of training step, confirms this interpretation. Early in training, evaluation reward is relatively high (around 0.70 or above) but gradually declines to approximately 0.25 as training proceeds. The combination of a decreasing NLL and a decreasing evaluation reward suggests that PPO is over-optimizing the surrogate objective. The model becomes overconfident in suboptimal responses, illustrating a classic failure mode where the surrogate loss diverges from the true task reward.

Over time, PPO leads to overly brief captions (e.g. ”a boy” , ”a girl”) leading to severely less informative captions and degrading performance. This behavior underscores a key lesson: in sparse-reward RL settings with small models and limited data, PPO does not automatically guarantee improved performance. Careful monitoring of both loss and reward, and potentially early stopping or stronger regularization, are necessary to prevent degradation.

C. GRPO and REINFORCE Training Curves

In contrast to PPO, GRPO and REINFORCE exhibit very different training dynamics.

Figure 4 plots the evaluation mean reward for the best GRPO run. The curve oscillates between approximately 0.58 and 0.73 across training, without the clear downward drift observed with PPO. This oscillatory but consistently high reward suggests that GRPO maintains exploration while still concentrating probability mass on useful answers. The group-relative ranking mechanism appears to protect the policy from prematurely collapsing into overconfident but incorrect choices.

Figures 5 and 6 show the GRPO training mean reward and NLL loss respectively. The reward curve displays moderate

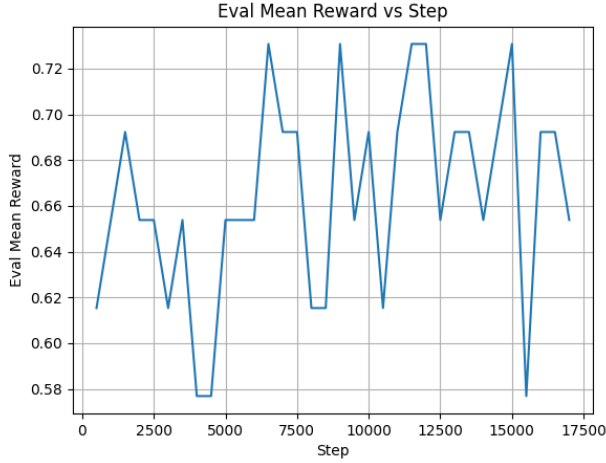


Fig. 4. GRPO evaluation mean reward vs. training steps.

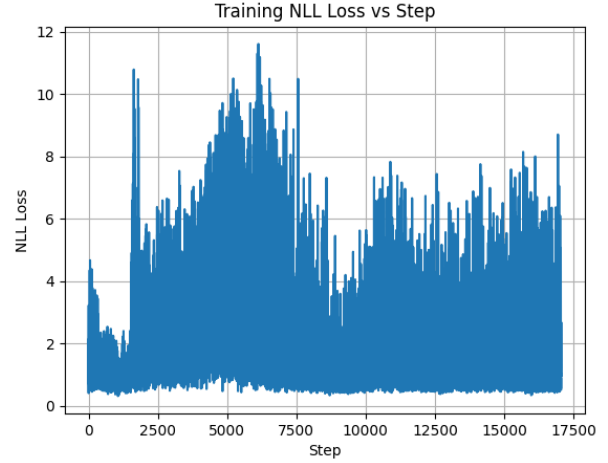


Fig. 6. Training NLL loss over GRPO steps.

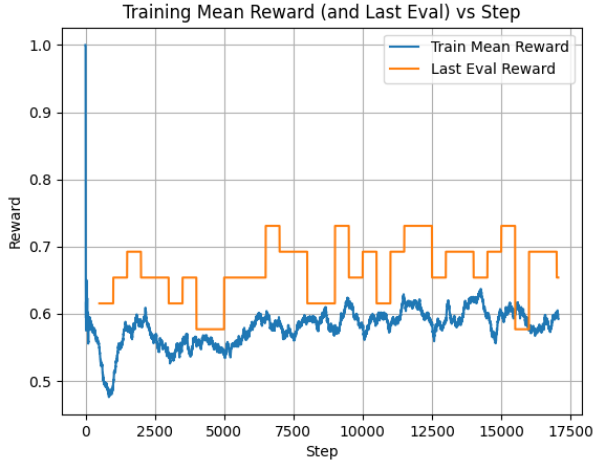


Fig. 5. Training mean reward over GRPO steps.

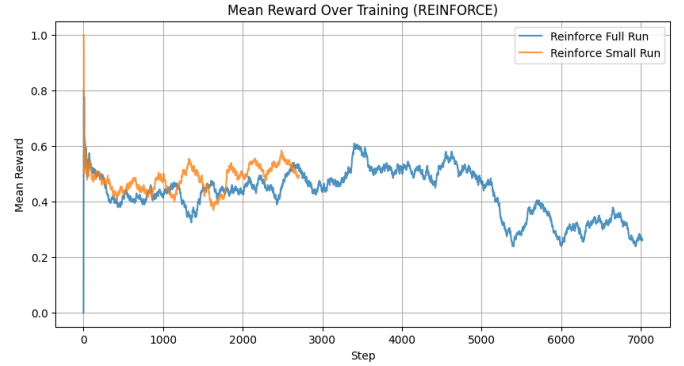


Fig. 7. REINFORCE mean reward (full vs. short runs).

fluctuations but remains centered around 0.55–0.60, while the NLL loss decreases but stabilizes at a higher level than PPO’s final NLL. In other words, GRPO prefers a policy that is less confident overall but more accurate in expectation. This trade-off aligns with its objective of optimizing relative advantages rather than absolute log-likelihood.

Finally, Figure 7 compares mean reward across a full REINFORCE run and a shorter exploratory run. The reward frequently dips toward 0.25—the expected reward of a uniform random policy over four options—and eventually fails to recover. Qualitative inspection of outputs shows that reward-hacking leaves the model’s text to degenerate into repetitive tokens, confirming that REINFORCE is unable to maintain a useful policy under such high-variance gradients and sparse rewards.

D. Qualitative Examples

Quantitative metrics provide only a partial view of model behavior. To better understand how the different algorithms affect answer quality and style, a set of qualitative examples was analyzed. The following summary condenses the detailed examples from the evaluation script.

1) *Sample 0: Baseball Player Swinging a Bat*: The image depicts a baseball player on a field preparing to swing a bat. The MCQ asks, “What is the player doing in the image?” with options including catching a ball, running, swinging a bat, and pitching. The correct answer is “swinging a bat.”

- **PPO** correctly selects option C and generates a concise caption like “A baseball player holding a bat on a field.” This is minimal but accurate.
- **GRPO** also selects option C but produces a far richer description, mentioning the striped uniform, helmet, spectators in the background, and the grassy infield. This suggests that GRPO implicitly preserves more visual detail while still solving the MCQ.
- **REINFORCE** selects an incorrect option and outputs a long, degenerate sequence of repeated tokens (“copy, copy, copy, ...”), indicative of catastrophic collapse.

2) *Sample 1: Player Uniform Number*: For a question about the uniform number of the same player, both PPO and GRPO predict an incorrect option, and REINFORCE again degenerates. This example highlights a limitation of the underlying model capacity and dataset: precise numeric attributes are difficult to infer, and the RL signal is too weak to teach such fine-grained recognition.

3) *Sample 2: Player Position Category*: When asked about the player’s general position (e.g., pitcher, catcher, base runner, or “baseball player”), all three methods occasionally guess option D correctly. Even REINFORCE can answer correctly when the options include a generic category, though its textual output is still degenerate.

4) *Sample 3: Scene Atmosphere*: For a question about the scene’s overall atmosphere (e.g., “calm and peaceful” vs. “excited and energetic”), PPO and GRPO both tend to pick the correct “excited and energetic” option, likely exploiting contextual clues from crowds and action poses. REINFORCE frequently misclassifies the atmosphere as “dull and uninteresting,” further undercutting its reward.

5) *Sample 4: Number of Students in a Library*: In an image showing several students in a library with pizza, the MCQ asks for the number of students. GRPO is the only method that reliably predicts the correct count, whereas PPO over- or underestimates and REINFORCE again collapses. This aligns with the aggregate accuracy results: GRPO appears more capable of consolidating disparate visual details into a coherent answer.

Overall, the qualitative analysis supports the quantitative findings: GRPO tends to produce detailed, grounded descriptions and correct answers; PPO produces shorter but often correct answers with some overfitting; REINFORCE degrades into repetitive, uninformative text.

IV. CODE STATUS AND RESOURCES USED

Our entire codebase can be found at : https://github.com/Rav7an/cs258_CapRL . All experiments were conducted on Google Colab Pro using A100 GPU and mixed-precision training where supported by the Qwen2-VL implementation. The key resource characteristics are:

- **Hardware**: T4 GPU for MCQ generation, and A100, H100 GPUs for the actual RL methods
- **Runtime**: PPO runs typically required 14–15 hours to reach 12,000 steps; GRPO runs required around 13–14 hours to reach around 15K steps; the full REINFORCE run extended to 13,000 steps and took roughly 7–8 hours.
- **Storage**: Model checkpoints and logs occupied several gigabytes on Google Drive.

The codebase is structured to be self-contained: once the dataset and model weights are downloaded, all scripts can be rerun from scratch with the same configuration files. The primary limitations encountered were Colab timeouts and VRAM limits, which occasionally truncated longer exploratory runs.

V. CONCLUSION AND FUTURE DIRECTIONS

This work demonstrates that it is possible to adapt the conceptual ideas of CapRL to an open, small-model setting focused on visual MCQ reasoning. By constructing a fully synthetic MCQ dataset from ShareGPT4V captions and training a 2B-parameter Qwen2-VL model using REINFORCE, PPO, and GRPO, the project provides a concrete case study of sparse-reward RL for multimodal reasoning.

Among the algorithms tested, GRPO delivers the best balance between stability and performance, achieving 63% accuracy and maintaining high evaluation rewards throughout training. PPO initially performs well but eventually over-optimizes its surrogate objective, as evidenced by a decreasing NLL combined with a declining reward. This highlights a subtle but important discrepancy between likelihood improvement and actual task performance when the reward is sparse and the model capacity is limited. REINFORCE, lacking any variance-reduction or clipping mechanisms, fails to produce a reliable policy and frequently collapses.

Several avenues for future work emerge naturally:

- **Reward shaping and curriculum design**. Instead of a single binary reward, intermediate shaping signals (e.g., partial credit for semantically close answers or auxiliary captioning losses) could improve learning speed and stability.
- **Multi-step reasoning tasks**. Extending the environment to multi-turn dialogues or chain-of-thought reasoning would more closely reflect real-world multimodal tasks and could better exploit RL’s sequential nature.
- **Larger but still open models**. As open-weight VLMs in the 7B–13B range become easier to deploy, repeating these experiments at higher capacity might yield more decisive gains and allow more challenging MCQs.
- **Human-in-the-loop feedback**. Replacing purely synthetic MCQs with human-written or human-curated questions could reduce dataset artifacts and align the reward signal more closely with real evaluation criteria.

Overall, the project provides both a working open-source baseline and a set of empirical insights about the behavior of PPO, GRPO, and REINFORCE on visual MCQ reasoning, thereby laying the groundwork for more ambitious CapRL-style research on modest hardware.

REFERENCES

- [1] CapRL: Reinforcement Learning for Visually Grounded Captioning, 2024.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” 2017.
- [3] Group Relative Policy Optimization, 2024.
- [4] R. J. Williams, “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.